Proyecto de Investigación y Desarrollo (I+D) — Bases de Datos

WheelsOn

Una red profesional sobre una base de datos de grafos

Diseño e implementación de una aplicación basada en una base de datos de grafos para la gestión y el análisis de relaciones en una red profesional, construida con Neo4j y Django.

Dubal Antonio Aguilar Torres	200207378
Carlos Miguel Calabria Patiño	200209929
Angelo David de Leon Caneda	200205958
Mateo José Cisneros Galeano	200205979
Luis Angel Cuellar Ballesteros	200163979

Docente:

Wilson Manuel Estrada Bernal

Ingeniería de Sistemas
Universidad del Norte
Barranquilla, Colombia

Noviembre de 2026

Índice

1. Introducción **2**

2. Bases de datos de grafos **2**
2.1. La información como una red . . . 2
2.2. Por qué recorrer un grafo es rápido . . . 3
2.3. Cypher, el lenguaje de consultas . . . 3

3. La aplicación **3**

4. Modelo utilizado **5**
4.1. Visión general . . . 5
4.2. Nodos . . . 5
4.3. Relaciones . . . 6
4.4. Decisiones de diseño . . . 7
4.5. Restricciones e índices . . . 9

5. Scripts de inicio de la base de datos **9**
5.1. Carga de datos . . . 9
5.2. Cómo se ejecutan . . . 10

6. Consultas utilizadas **10**
6.1. Usuarios con habilidades similares . . . 10
6.2. Contactos en común . . . 11
6.3. Ruta más corta entre un usuario y una empresa . . . 11
6.4. Candidatos para una oferta según conexiones y competencias . . . 12
6.5. Proyectos que comparten tecnologías o miembros relacionados . . . 13
6.6. Habilidades más demandadas . . . 14
6.7. Recomendaciones . . . 14
6.8. Panel de analítica . . . 15

7. Investigación: ¿cuándo conviene un grafo y cuándo no? **16**
7.1. WheelsOn en un modelo relacional . . . 16
7.2. Dónde gana el grafo . . . 17
7.3. Dónde no gana el grafo . . . 18
7.4. Resumen . . . 19

8. Conclusiones **20**

9. Referencias **20**

A. Script de inicio de la base de datos **20**

B. Script de consultas **22**

1. Introducción

Desarrollamos WheelsOn, una red profesional en la que cada persona tiene un perfil, se conecta con otras, declara sus habilidades, trabaja en empresas, participa en proyectos y se postula a ofertas de empleo. En una red así importa más cómo se relacionan los datos que cada dato por separado: quién conoce a quién, qué habilidades comparten dos personas o por qué contactos se puede llegar a una empresa.

Por eso guardamos toda la información de la red en una base de datos de grafos, Neo4j, y construimos la aplicación web en Python con Django.

En este documento explicamos primero qué es una base de datos de grafos (Sección 2) y presentamos la aplicación (Sección 3). Luego describimos el modelo que diseñamos para WheelsOn (Sección 4), los scripts con los que creamos la base y cargamos los datos (Sección 5) y las consultas con las que respondemos las preguntas del enunciado (Sección 6). Por último, a partir de lo que aprendimos al construir WheelsOn, analizamos en qué escenarios el modelo de grafos supera al relacional y en cuáles no (Sección 7).

2. Bases de datos de grafos

2.1. La información como una red

Una base de datos de grafos guarda la información como una red de puntos unidos por líneas. Los puntos se llaman **nodos** y representan cosas: una persona, una empresa, una habilidad. Las líneas se llaman **relaciones** y dicen cómo se vinculan esas cosas, por ejemplo, "Valentina trabaja en Andina Logística".

Neo4j usa el *modelo de grafo de propiedades*, que tiene cuatro piezas:

Tabla 1. Piezas del modelo de grafo de propiedades.

Pieza	Qué es	Ejemplo en WheelsOn
Nodo	Una entidad.	Valentina Ríos
Etiqueta	El tipo de un nodo.	`:Usuario`, `:Empresa`
Relación	Un vínculo entre dos nodos, con tipo y dirección.	**`TRABAJA_EN`**
Propiedad	Un dato guardado en un nodo o en una relación.	`nombre`, `cargo`

A diferencia de otros modelos, aquí las relaciones también guardan datos. En la Figura 1, el cargo y la fecha de inicio están en la relación **`TRABAJA_EN`**, porque son datos del vínculo entre Valentina y la empresa.

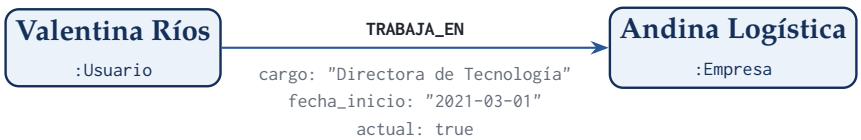


Figura 1. Dos nodos y una relación con propiedades, tomados de los datos de demostración.

2.2. Por qué recorrer un grafo es rápido

En una base relacional, los vínculos entre registros se reconstruyen en cada consulta: el motor busca, tabla por tabla, las filas cuyas claves coinciden (la operación `JOIN`). Cada nivel más de profundidad en la pregunta agrega otro `JOIN`.

En Neo4j, cada nodo guarda referencias directas a sus relaciones. Pasar de un nodo a sus vecinos es seguir esas referencias, sin buscar en una tabla. Por eso el costo de un recorrido depende de cuántos nodos visita, no de cuántos datos hay en total en la base. Esta propiedad se conoce como *adyacencia sin índice*.

2.3. Cypher, el lenguaje de consultas

Neo4j se consulta con Cypher. Una consulta describe la forma que deben tener los datos con un dibujo hecho de texto: los nodos van entre paréntesis y las relaciones entre corchetes, con una flecha que indica la dirección. Esta consulta devuelve quiénes trabajan en Andina Logística y con qué cargo:

```
MATCH (u:Usuario)-[t:TRABAJA_EN]->(e:Empresa {nombre: 'Andina Logística'})
RETURN u.nombre, t.cargo
```

`MATCH` describe el patrón que se busca y `RETURN` lo que se quiere obtener. Además, en Cypher la cantidad de saltos puede ser variable: `-[:CONECTA_CON*..6]-` significa “por una cadena de entre uno y seis contactos”. Con eso se escriben en una línea preguntas como la ruta más corta entre dos personas.

3. La aplicación

Construimos WheelsOn como una aplicación web en la que cada pantalla lee y escribe directamente en Neo4j.

Tabla 2. Tecnologías utilizadas.

Parte	Tecnología
Base de datos	Neo4j 5.26, en un contenedor de Docker. Django usa además un archivo SQLite local solo para las sesiones de inicio de sesión; ningún dato de la red se guarda ahí.
Aplicación web	Python 3.12 y Django 6.
Conexión con Neo4j	*neomodel*, que convierte nodos en objetos de Python, y consultas Cypher escritas a mano para todo lo que recorre el grafo.
Interfaz	HTML con Tailwind y Alpine.js; el grafo interactivo usa vis-network y las gráficas, Chart.js.

En la Tabla 3 ubicamos cada funcionalidad del enunciado dentro de la aplicación.

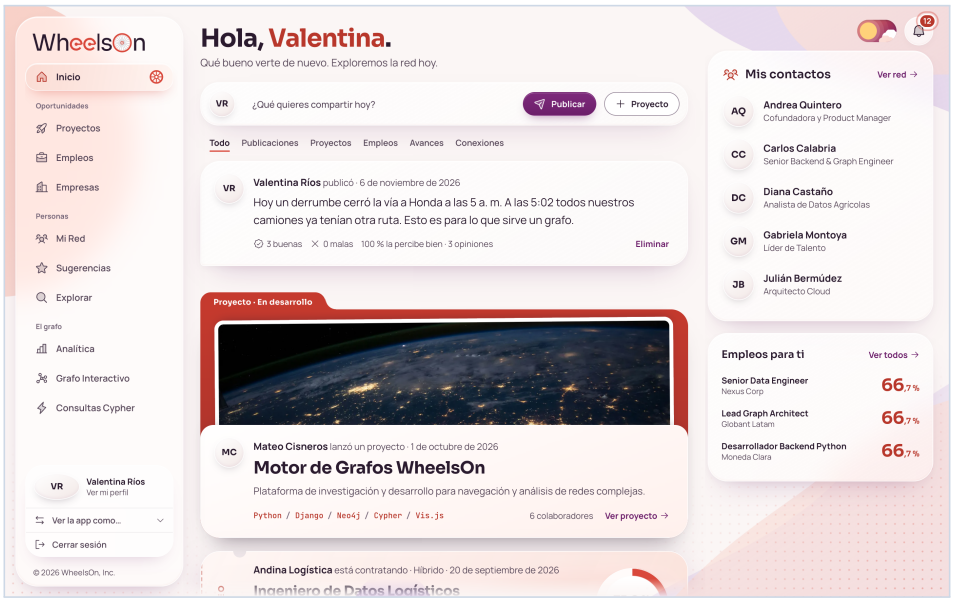


Figura 2. Pantalla de inicio de WheelsOn con la sesión de Valentina Ríos, una de las personas de los datos de demostración.

Tabla 3. Funcionalidades pedidas y dónde se encuentran.

Funcionalidad	En la aplicación
CRUD de usuarios	Registro, perfil, edición y eliminación de la cuenta.
Conexiones entre usuarios	Solicitudes de conexión que la otra persona acepta o rechaza.
Vincular usuarios con habilidades, empresas y proyectos	Desde el perfil, con nivel, cargo o rol. Los empleos y la entrada a proyectos los aprueba la empresa o el líder.
Empresas y sus ofertas	Páginas de Empresas y Empleos.
Habilidades requeridas por una oferta	Requisitos de cada oferta, con nivel mínimo y prioridad.
Postularse a una oferta	Botón *Postularme*. La empresa mueve la postulación por etapas hasta aceptarla o rechazarla.
Caminos y conexiones	Consultas Cypher: rutas entre personas, hacia empresas y hacia ofertas (Sección 6).
Recomendaciones	Sugerencias de contactos, empleos, colaboradores para proyectos y talento para empresas.
Panel de analítica	Influencia, comunidades y habilidades más demandadas (Sección 6.8).

4. Modelo utilizado

4.1. Visión general

Nuestro modelo tiene 8 tipos de nodo y 23 tipos de relación (Figura 3). Conservamos los nombres que propone el enunciado y agregamos los nodos y relaciones que hicieron falta para que la aplicación funcionara como una red real: publicaciones, avances de proyecto, notificaciones, permisos, alianzas entre empresas y solicitudes pendientes.

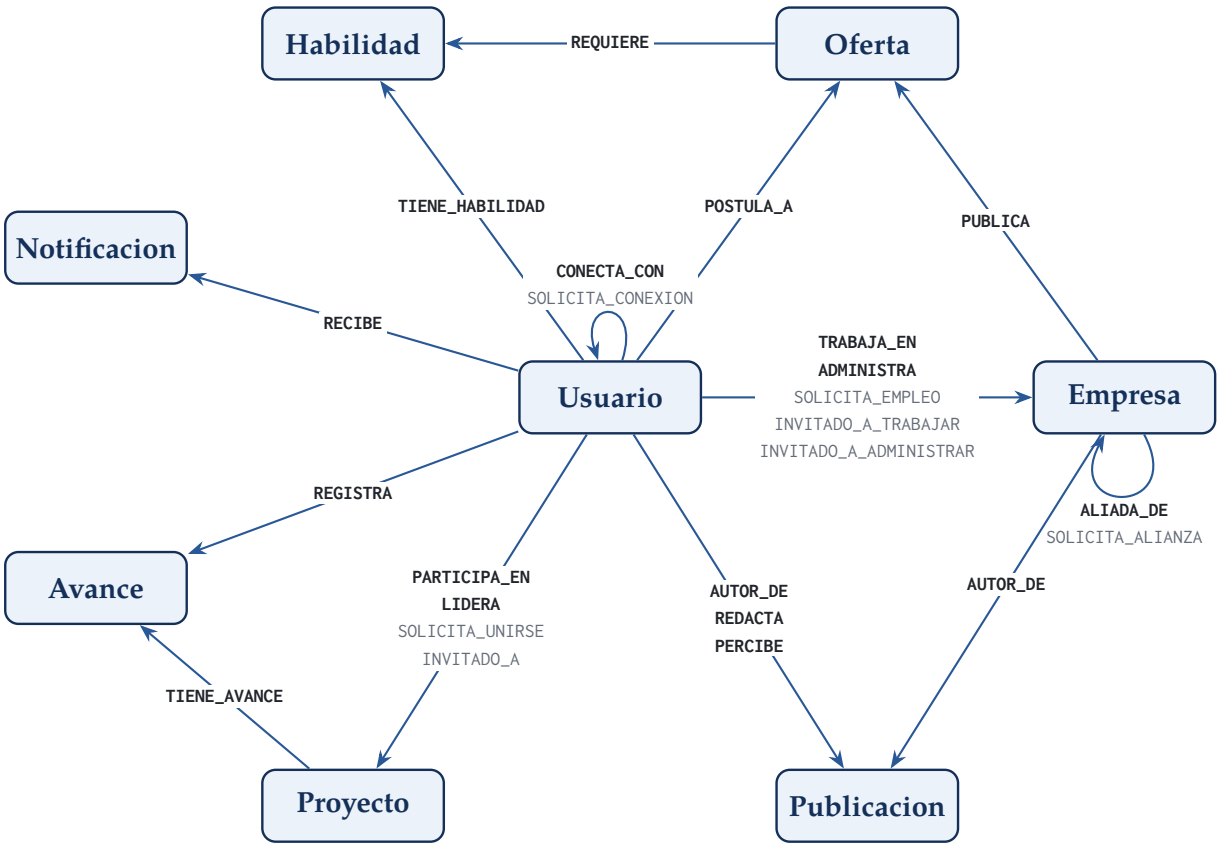


Figura 3. Modelo de WheelsOn: los 8 tipos de nodo y los 23 tipos de relación. En gris, las solicitudes e invitaciones pendientes, que se reemplazan por la relación definitiva cuando se aceptan (Tabla 6).

4.2. Nodos

A cada nodo le dimos un identificador propio (`uid`, `eid`, `oid`, etc.) que no se repite. Es el que aparece en las direcciones de la aplicación, por ejemplo `/usuarios/usr_valentina16/`.

Tabla 4. Tipos de nodo.

Etiqueta	Representa	Propiedades principales
`:Usuario`	Una persona registrada.	`uid`, `nombre`, `apellido`, `email`, `titulo_profesional`, `ubicacion`, `rol`
`:Empresa`	Una organización.	`eid`, `nombre`, `industria`, `tamano`, `ubicacion`
`:Habilidad`	Un conocimiento o competencia.	`hid`, `nombre`, `categoria` (técnica, blanda o idioma)
`:Proyecto`	Un trabajo o iniciativa.	`pid`, `nombre`, `estado`, `tecnologias` (lista), fechas
`:Oferta`	Una vacante laboral.	`oid`, `titulo`, `modalidad`, `salario_min`, `salario_max`, `cupos`, `estado`
`:Publicacion`	Lo que alguien comparte en el inicio.	`pubid`, `texto`, `fecha`
`:Avance`	Una novedad de un proyecto.	`aid`, `texto`, `estado`, `fecha`
`:Notificacion`	Un aviso para una persona.	`nid`, `titulo`, `detalle`, `fecha`

4.3. Relaciones

Tabla 5. Relaciones del modelo.

Relación	De → a	Propiedades	Significado
`CONECTA_CON`	Usuario – Usuario	`fecha`, `mensaje`, `estado` (siempre *aceptada*)	Contacto confirmado por ambas personas. Hay una sola relación por pareja y las consultas la recorren sin importar la dirección.
`TIENE_HABILIDAD`	Usuario → Habilidad	`nivel`, `anios_experiencia`	La persona domina esa habilidad.
`TRABAJA_EN`	Usuario → Empresa	`cargo`, `fecha_inicio`, `fecha_fin`, `actual`	Empleo actual o anterior.
`ADMINISTRA`	Usuario → Empresa	—	Quién gestiona la empresa y sus ofertas.
`PARTICIPA_EN`	Usuario → Proyecto	`rol`, `fecha_inicio`, `activo`	Miembro del equipo de un proyecto.
`LIDERA`	Usuario → Proyecto	—	Quién gestiona el proyecto y su equipo.
`PUBLICA`	Empresa → Oferta	`fecha_publicacion`	La empresa ofrece esa vacante.
`REQUIERE`	Oferta → Habilidad	`nivel_minimo`, `prioridad`	Requisito de la vacante, obligatorio o deseable.
`POSTULA_A`	Usuario → Oferta	`estado`, `fecha`, `fecha_estado`, `carta_presentacion`	Postulación y la etapa en que va.
`ALIADA_DE`	Empresa – Empresa	`fecha`	Alianza aceptada entre dos empresas.
`AUTOR_DE`	Usuario o Empresa → Publicacion	—	A nombre de quién aparece la publicación.

Relación	De → a	Propiedades	Significado
`REDACTA`	Usuario → Publicacion	—	Qué persona escribió una publicación de empresa.
`PERCIBE`	Usuario → Publicacion	`tipo` (buena o mala), `fecha`	Reacción a una publicación.
`TIENE_AVANCE`	Proyecto → Avance	—	Historial del proyecto.
`REGISTRA`	Usuario → Avance	—	Quién registró el avance.
`RECIBE`	Usuario → Notificacion	—	Avisos de la persona.

Tabla 6. Solicitudes e invitaciones pendientes. Al aceptarse, cada una se reemplaza por la relación definitiva.

Pendiente	De → a	Al aceptarse se convierte en
`SOLICITA_CONEXION`	Usuario → Usuario	`CONECTA_CON`
`SOLICITA_EMPLEO`	Usuario → Empresa	`TRABAJA_EN`, cuando la empresa confirma el empleo.
`INVITADO_A_TRABAJAR`	Usuario → Empresa	`TRABAJA_EN`, cuando la persona acepta.
`INVITADO_A_ADMINISTRAR`	Usuario → Empresa	`ADMINISTRA`: la administración pasa a esa persona.
`SOLICITA_UNIRSE`	Usuario → Proyecto	`PARTICIPA_EN`
`INVITADO_A`	Usuario → Proyecto	`PARTICIPA_EN` o `LIDERA`, según la invitación.
`SOLICITA_ALIANZA`	Empresa → Empresa	`ALIADA_DE`

4.4. Decisiones de diseño

Los datos de un vínculo van en la relación

Pusimos el cargo en `TRABAJA_EN`, la etapa de la postulación en `POSTULA_A` y la prioridad de un requisito en `REQUIERE`. Si el cargo estuviera en el nodo de la persona, nadie podría tener dos empleos.

Lo pendiente vive en relaciones aparte

Guardamos una solicitud de conexión como una relación `SOLICITA_CONEXION`. Si se acepta, la borramos y creamos `CONECTA_CON`; si se rechaza, solo la borramos. Así, las rutas, las recomendaciones y la analítica recorren únicamente `CONECTA_CON` y nunca cuentan como contacto a alguien que no ha respondido. Aplicamos la misma idea a empleos, proyectos, administración y alianzas (Tabla 6).

Una postulación avanza por etapas

`POSTULA_A` pasa por *enviada*, *en revisión* y *entrevista*, hasta terminar en *aceptada*, *rechazada* o *no seleccionada*. Aceptar a alguien crea su relación `TRABAJA_EN` con el título del empleo como cargo. Cuando se llenan los cupos, la oferta se cierra y quienes seguían en proceso quedan como *no seleccionada*. Los empleos terminados los conservamos con `actual: false` para la trayectoria de la persona, pero en las rutas solo usamos los actuales.

Los permisos también están en el grafo

Decidimos quién puede editar algo con una relación. Para cambiar una oferta, por ejemplo, la aplicación comprueba que exista este camino (o que la persona sea administradora de la plataforma):

```
(persona)-[:ADMINISTRA]->(empresa)-[:PUBLICA]->(oferta)
```

Las notificaciones son nodos

Lo que está pendiente se deduce de las relaciones de la Tabla 6. Pero cuando alguien rechaza una solicitud, la relación se borra y ya no queda nada de qué avisar. Por eso guardamos cada aviso como un nodo `:Notificacion` unido a la persona con **`RECIBE`**.

Lo que dejamos como propiedad

Guardamos las tecnologías de un proyecto como una lista de texto (`tecnologias`), no como relaciones con nodos `:Habilidad`. Esto simplificó la creación de proyectos, pero nos obliga a comparar texto cuando buscamos proyectos parecidos (Sección 6.5). Como mejora, proponemos una relación `USA_TECNOLOGIA` hacia `:Habilidad`.

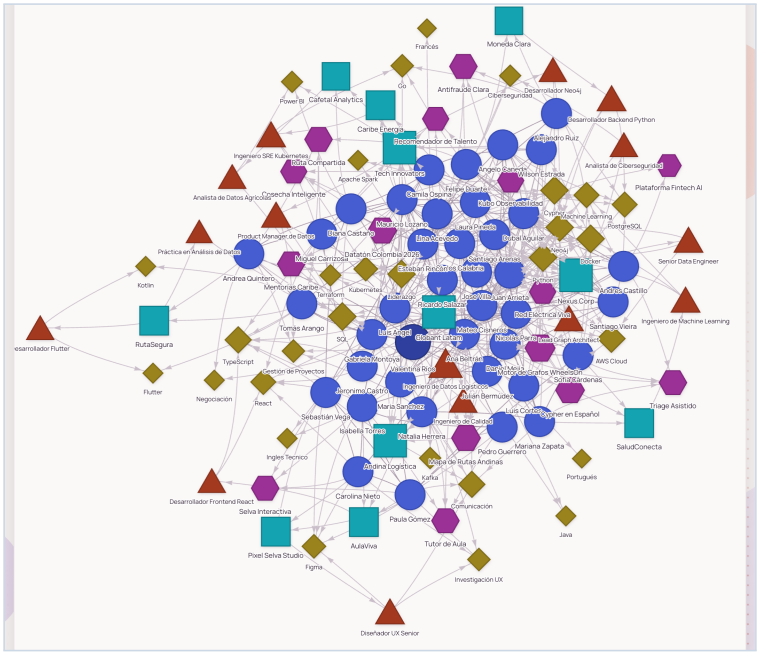


Figura 4. Grafo interactivo de la aplicación: los datos de demostración tal como están en Neo4j. Círculos azules: usuarios; cuadrados turquesa: empresas; rombos: habilidades; hexágonos morados: proyectos; triángulos rojos: ofertas.

4.5. Restricciones e índices

Las **restricciones de unicidad** impiden que se repita un valor. Hay once: el identificador de cada uno de los ocho tipos de nodo, el correo de cada usuario y los nombres de empresas y habilidades. Los **índices** aceleran las búsquedas frecuentes. Hay cinco: el nombre y apellido de los usuarios, la industria de las empresas, la categoría de las habilidades y el estado de las ofertas y de los proyectos.

Neo4j no puede exigir por sí mismo el resto de las reglas del negocio, como que una oferta pertenezca a una sola empresa o que no se acepte a más personas que cupos. Esas las hicimos cumplir desde la aplicación, dentro de transacciones (Sección 7.3).

5. Scripts de inicio de la base de datos

Organizamos los scripts en la carpeta `scripts/` del repositorio.

Tabla 7. Scripts de la base de datos, en el orden en que se ejecutan.

Archivo	Contenido
`constraints.cypher`	Las 11 restricciones y los 5 índices de la Sección 4.5. Usa `IF NOT EXISTS`, así que se puede ejecutar varias veces sin error. Va primero (completo en el Anexo A).
`init_db.cypher`	Lo mismo, una sentencia por línea, para pegarlo en Neo4j Browser.
`seed_data.cypher`	Datos base: 15 personas, 3 empresas, 12 habilidades, 3 proyectos y 2 ofertas, con sus relaciones.
`datos_demo.cypher`	La red de demostración: 25 personas, 8 empresas y 13 ofertas más, publicaciones, avances, postulaciones en todas las etapas y solicitudes pendientes.
`queries.cypher`	Las consultas de la Sección 6 (completo en el Anexo B).

5.1. Carga de datos

Primero creamos los nodos y después las relaciones. Para crear una relación, buscamos sus dos extremos por su identificador con `MATCH` y los unimos con `CREATE`:

```
CREATE (:Empresa {eid: "emp_andina04", nombre: "Andina Logística",
                  industria: "Logística", tamano: "mediana", ...});

MATCH (a:Usuario {uid: "usr_valentina16"}), (b:Empresa {eid: "emp_andina04"})
CREATE (a)-[:TRABAJA_EN {cargo: "Directora de Tecnología",
                         fecha_inicio: "2021-03-01", actual: true}]->(b);
```

Guardamos las fechas como texto con el formato `AAAA-MM-DD`, que es como *neomodel* las lee y escribe; con ese formato, el orden alfabético coincide con el cronológico. Las contraseñas las guardamos cifradas, con el mismo método que usa Django.

Con los dos scripts de datos, la base queda con **175 nodos y 788 relaciones**.

5.2. Cómo se ejecutan

Incluimos en la aplicación un comando que ejecuta los tres scripts en orden:

```
python manage.py cargar_datos             # en una base vacía
python manage.py cargar_datos --limpiar   # borra todo y vuelve a cargar
```

También se pueden ejecutar a mano con `cypher-shell`, en el mismo orden: primero `constraints`, luego `seed_data` y al final `datos_demo`. Los pasos completos para instalar y correr el proyecto están en el archivo `README.md` del repositorio.

6. Consultas utilizadas

Reunimos en `queries.cypher` 17 consultas: las que responden las seis preguntas del enunciado, las de caminos, las de recomendación y las que alimentan la analítica y el grafo interactivo. Las consultas 1 a 14 tienen la misma lógica que las que ejecuta la aplicación; la diferencia es que en el script devuelven columnas legibles, como nombres y títulos, para poder probarlas en Neo4j Browser, mientras que la aplicación pide los nodos completos. Las consultas 15 a 17 son de apoyo, para revisar la base. El archivo completo está en el Anexo B.

Las palabras que empiezan con `$`, como `$uid`, son parámetros: la aplicación pone ahí el usuario, la empresa o la oferta elegida en pantalla.

Tabla 8. Preguntas del enunciado y la consulta que responde cada una.

Pregunta	Consulta	Sección
¿Qué usuarios tienen habilidades similares?	3	6.1
¿Qué contactos en común existen entre dos profesionales?	2	6.2
¿Cuál es la ruta más corta entre un usuario y una empresa?	4	6.3
¿Qué candidatos podrían recomendarse para una oferta?	6	6.4
¿Qué proyectos comparten tecnologías o miembros relacionados?	14	6.5
¿Qué habilidades son las más demandadas en las ofertas?	5	6.6

6.1. Usuarios con habilidades similares

Partimos de una persona, pasamos a sus habilidades y de ahí a las demás personas que tienen esas mismas habilidades. Cuantas más habilidades compartan, más arriba las ubicamos.

```
MATCH (u1:Usuario {uid: $uid1})-[:TIENE_HABILIDAD]->(h:Habilidad)
      <-[:TIENE_HABILIDAD]-(u2:Usuario)
WHERE u1 <> u2
WITH u2, count(h) AS total_compartidas, collect(h.nombre) AS habilidades_comunes
ORDER BY total_compartidas DESC, u2.nombre ASC
RETURN u2.uid AS uid, u2.nombre + ' ' + u2.apellido AS nombre_completo,
       total_compartidas, habilidades_comunes
LIMIT $limit;
```

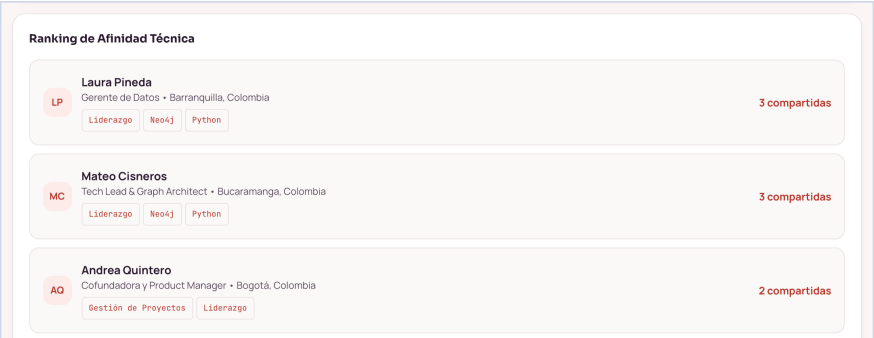


Figura 5. Personas con habilidades parecidas a las de Valentina Ríos.

6.2. Contactos en común

Un contacto en común es alguien conectado con las dos personas a la vez. El patrón dibuja exactamente eso: una persona en el medio unida a ambas con **CONECTA_CON**. Escribimos las relaciones sin flecha porque el contacto es mutuo.

```
MATCH (u1:Usuario {uid: $uid1})-[:CONECTA_CON]-(comun:Usuario)
      -[:CONECTA_CON]-(u2:Usuario {uid: $uid2})
WHERE u1 <> u2
RETURN DISTINCT comun.uid AS uid,
       comun.nombre + ' ' + comun.apellido AS nombre_completo,
       comun.titulo_profesional AS titulo
ORDER BY nombre_completo ASC;
```

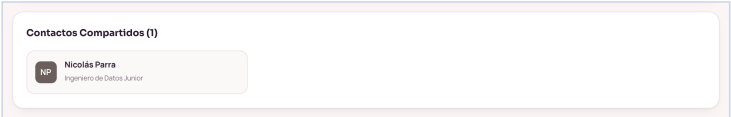


Figura 6. Valentina Ríos y Laura Pineda tienen un contacto en común.

6.3. Ruta más corta entre un usuario y una empresa

Usamos `shortestPath` para buscar la cadena más corta, de hasta seis pasos, que une a la persona con la empresa. Permitimos pasar por contactos (**CONECTA_CON**) y por empleos (**TRABAJA_EN**), pero solo por empleos actuales: alguien que ya no trabaja en una empresa no sirve como puente hacia ella.

```
MATCH (u:Usuario {uid: $uid}), (e:Empresa {eid: $eid})
MATCH p = shortestPath((u)-[:CONECTA_CON|TRABAJA_EN*..6]-(e))
WHERE all(r IN relationships(p)
          WHERE type(r) <> 'TRABAJA_EN' OR coalesce(r.actual, true))
RETURN [n IN nodes(p) | {tipo: labels(n)[0], id: coalesce(n.uid, n.eid),
        nombre: CASE WHEN n:Usuario THEN n.nombre + ' ' + n.apellido
                     ELSE n.nombre END}] AS pasos_ruta,
       length(p) AS longitud_camino;
```

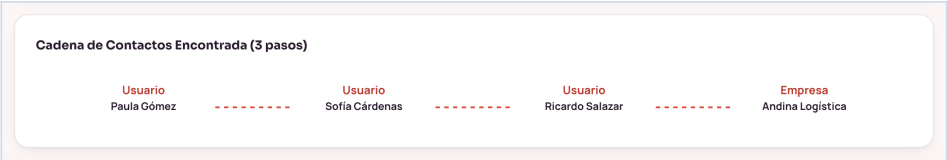


Figura 7. Paula Gómez llega a Andina Logística en tres pasos: conoce a Sofía Cárdenas, que conoce a Ricardo Salazar, que trabaja en la empresa.

Con la misma idea escribimos la consulta 1, entre dos personas y usando solo `CONECTA_CON`, y la consulta 13, que extiende el camino hasta una oferta con la relación `PUBLICA` (Figura 8).



Figura 8. Camino de Paula Gómez hasta una oferta de Andina Logística: pasa por dos contactos y por la empresa, que publica la oferta.

6.4. Candidatos para una oferta según conexiones y competencias

Para responder esta pregunta combinamos las dos cosas que pide. Primero buscamos a las personas que cumplen al menos un requisito de la oferta (competencias). Después contamos cuántos de sus contactos trabajan en la empresa que la publica (conexiones). Con eso calculamos un puntaje:

$$\text{puntaje} = 3\times \text{requisitos que cumple} + 2\times \text{requisitos obligatorios que cumple} + 2\times \text{contactos en la empresa}$$

Dejamos fuera a quienes ya trabajan en esa empresa, porque no son candidatos.

```
MATCH (o:Oferta {oid: $oid})-[r:REQUIERE]->(h:Habilidad)<-[:TIENE_HABILIDAD]-(u:Usuario)
WHERE NOT EXISTS {
  MATCH (u)-[t:TRABAJA_EN]->(:Empresa)-[:PUBLICA]->(o) WHERE coalesce(t.actual, true)
}
WITH o, u, count(DISTINCT h) AS total_coincidencias,
     collect(DISTINCT h.nombre) AS habilidades_match,
     count(DISTINCT CASE WHEN r.prioridad = 'obligatoria' THEN h END) AS obligatorias_cubiertas
OPTIONAL MATCH (u)-[:CONECTA_CON]-(c:Usuario)-[t:TRABAJA_EN]->(:Empresa)-[:PUBLICA]->(o)
WHERE coalesce(t.actual, true)
WITH o, u, total_coincidencias, habilidades_match, obligatorias_cubiertas,
     collect(DISTINCT c.nombre + ' ' + c.apellido) AS contactos
RETURN u.uid AS uid, u.nombre + ' ' + u.apellido AS nombre_completo,
       total_coincidencias, obligatorias_cubiertas, habilidades_match, contactos,
       total_coincidencias * 3 + obligatorias_cubiertas * 2 + size(contactos) * 2 AS score,
       EXISTS { (u)-[:POSTULA_A]->(o) } AS ya_postulado
ORDER BY score DESC, total_coincidencias DESC, nombre_completo ASC
```

```
LIMIT $limit;
```

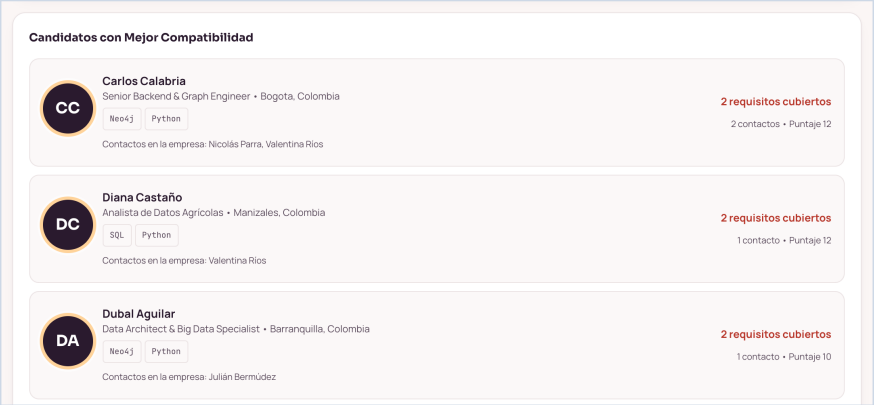


Figura 9. Mejores candidatos para *Ingeniero de Datos Logísticos*. Debajo de cada nombre aparecen las habilidades que cumple y sus contactos dentro de la empresa.

6.5. Proyectos que comparten tecnologías o miembros relacionados

Dado un proyecto, comparamos cada uno de los demás en tres aspectos y les damos un puntaje:

- **tecnologías en común** (2 puntos cada una), comparando las listas sin distinguir mayúsculas;
- **miembros en común**, personas que están en los dos equipos (3 puntos cada una);
- **miembros relacionados**, parejas de personas de cada equipo que están conectadas entre sí (1 punto cada pareja).

```
MATCH (p:Proyecto {pid: $pid}), (otro:Proyecto)
WHERE otro <> p
WITH p, otro, [t IN coalesce(p.tecnologias, [])
               WHERE toLower(t) IN [x IN coalesce(otro.tecnologias, []) | toLower(x)]]
              AS tecnologias
OPTIONAL MATCH (p)<-[:PARTICIPA_EN]-(m:Usuario)-[:PARTICIPA_EN]->(otro)
WITH p, otro, tecnologias, collect(DISTINCT m) AS comunes
OPTIONAL MATCH (p)<-[:PARTICIPA_EN]-(a:Usuario)-[:CONECTA_CON]-(b:Usuario)
               -[:PARTICIPA_EN]->(otro)
WHERE NOT a IN comunes AND NOT b IN comunes
WITH otro, tecnologias, comunes,
     collect(DISTINCT a.nombre + ' ' + a.apellido + ' - ' + b.nombre + ' ' + b.apellido)
     AS conectados
WITH otro, tecnologias, conectados, [m IN comunes | m.nombre + ' ' + m.apellido]
     AS miembros_comunes,
     size(tecnologias) * 2 + size(comunes) * 3 + size(conectados) AS score
WHERE score > 0
RETURN otro.pid AS pid, otro.nombre AS nombre, tecnologias, miembros_comunes,
       conectados, score
ORDER BY score DESC, nombre ASC
```

```
LIMIT $limit;
```

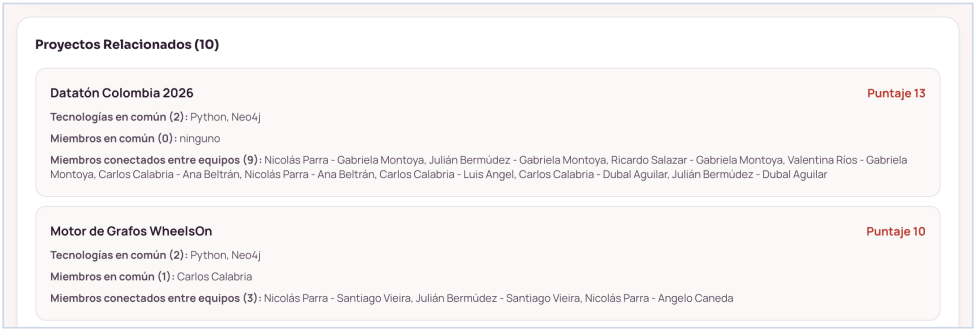


Figura 10. Proyectos relacionados con *Mapa de Rutas Andinas*.

6.6. Habilidades más demandadas

Contamos en cuántas ofertas abiertas aparece cada habilidad como requisito, separando las obligatorias de las deseables. Las ofertas cerradas no las tenemos en cuenta.

```
MATCH (h:Habilidad)<-[r:REQUIERE]-(o:Oferta {estado: 'abierta'})
RETURN h.hid AS hid, h.nombre AS habilidad, h.categoria AS categoria,
       count(o) AS total_ofertas,
       count(CASE WHEN r.prioridad = 'obligatoria' THEN 1 END) AS obligatorias,
       count(CASE WHEN r.prioridad = 'deseable' THEN 1 END) AS deseables
ORDER BY total_ofertas DESC, obligatorias DESC
LIMIT $limit;
```

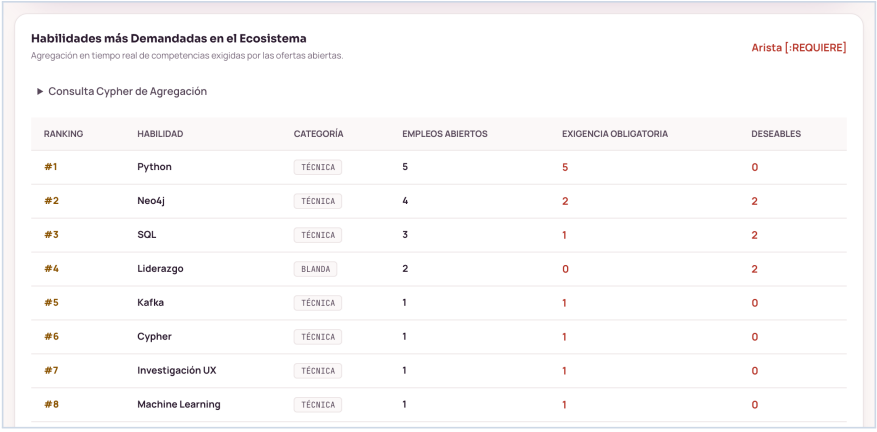


Figura 11. Habilidades más pedidas en las ofertas abiertas.

6.7. Recomendaciones

Con las consultas 7 a 10 generamos las sugerencias de la aplicación. Todas siguen la misma idea: partimos de un nodo, exploramos lo que tiene cerca y sumamos puntos por cada vínculo encontrado.

Tabla 9. Consultas de recomendación y cómo puntúan.

Recomienda	Consulta	Criterio
Contactos	7	Personas sin conexión directa: 4 puntos por contacto en común, 3 por empresa en común y 2 por habilidad en común.
Empleos	8	Ofertas abiertas a las que la persona no se ha postulado, ordenadas por el porcentaje de requisitos que ya cumple. Muestra también los que le faltan.
Colaboradores para un proyecto	9	5 puntos por cada habilidad que coincide con las tecnologías del proyecto y 3 por cada contacto en el equipo.
Talento para una empresa	10	4 puntos por cada habilidad pedida en sus ofertas abiertas y 3 por cada contacto que trabaja allí.

6.8. Panel de analítica

En el panel de analítica mostramos tres análisis de la red de contactos:

- **Usuarios más influyentes**, con tres medidas: cuántos contactos tiene cada persona (centralidad de grado), su PageRank y su papel como puente entre grupos (intermediación). El PageRank considera influyente a quien está conectado con personas que, a su vez, son influyentes.
- **Comunidades**, detectadas con el algoritmo de Louvain, que agrupa a las personas que tienen muchas conexiones entre sí y pocas hacia afuera. A cada comunidad la nombramos con las habilidades más frecuentes de sus miembros.
- **Habilidades más demandadas**: en cuántas ofertas abiertas se pide cada habilidad, como en la consulta 5, comparado con cuántas personas de la red la tienen.

Neo4j entrega la red completa con la consulta 11, y sobre ella aplicamos los algoritmos en Python:

```
MATCH (a:Usuario)-[:CONECTA_CON]->(b:Usuario)
RETURN a.uid, b.uid;
```

¿Por qué los algoritmos corren en Python y no en Neo4j?

Neo4j tiene una biblioteca para esto, *Graph Data Science*, pero hay que instalarla aparte. Preferimos que el proyecto funcione con una instalación normal de Neo4j en el computador de cualquier integrante. Con 40 personas, el cálculo en Python tarda una fracción de segundo. En una red grande convendría usar esa biblioteca, que hace el cálculo dentro de la base sin sacar los datos.

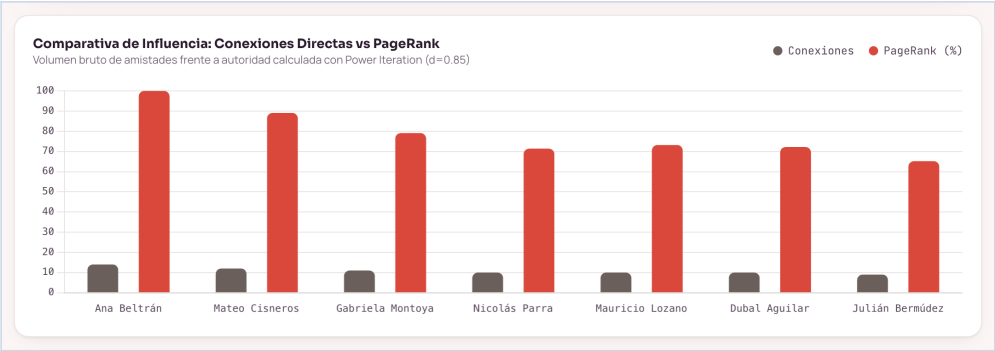


Figura 12. Las siete personas más influyentes según su PageRank, comparado con su número de contactos.

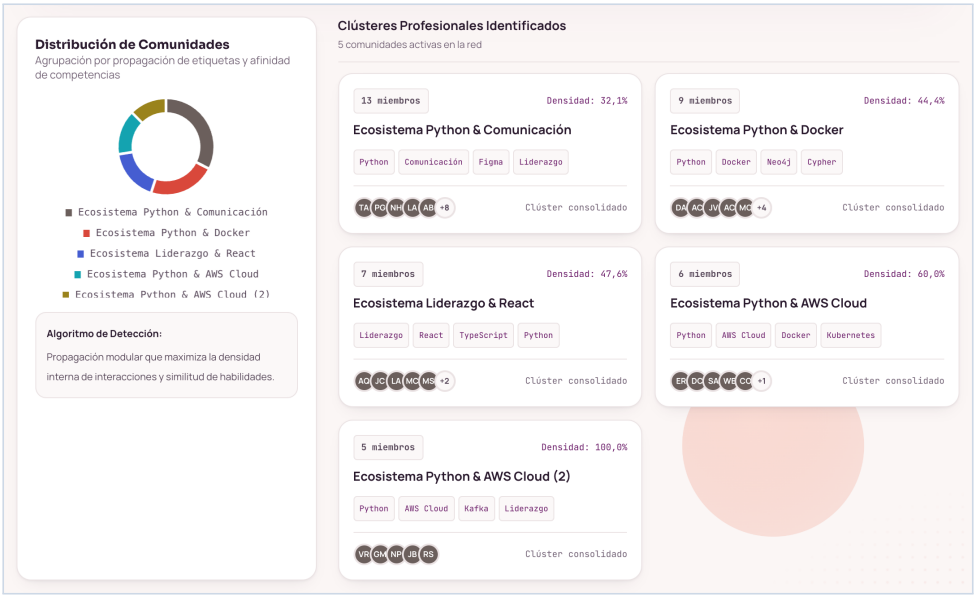


Figura 13. Las cinco comunidades que encuentra el algoritmo de Louvain en la red de demostración.

7. Investigación: ¿cuándo conviene un grafo y cuándo no?

7.1. WheelsOn en un modelo relacional

Para comparar, pensamos cómo se vería WheelsOn en tablas. Cada tipo de nodo sería una tabla (`usuario`, `empresa`, `oferta`...) y cada tipo de relación, una tabla intermedia con las claves de sus dos extremos y sus propiedades:

```
CREATE TABLE conexion (
  usuario_a TEXT REFERENCES usuario(uid),
  usuario_b TEXT REFERENCES usuario(uid),
  fecha     DATE,
  PRIMARY KEY (usuario_a, usuario_b)
);
```

Los dos modelos pueden guardar la misma información. Lo que cambia es el trabajo que hace cada una para responder, y eso depende del tipo de pregunta.

7.2. Dónde gana el grafo

Recorridos de profundidad variable

La ruta más corta entre dos personas no tiene una cantidad fija de pasos: pueden ser dos o seis. En Cypher son dos líneas (Sección 6.3). En SQL hace falta una consulta recursiva que, nivel por nivel, extiende todos los caminos posibles y lleva la cuenta de por dónde pasó cada uno para no dar vueltas:

```sql
WITH RECURSIVE ruta(actual, camino, saltos) AS (
  SELECT 'usr_valentina16', ARRAY['usr_valentina16'], 0
  UNION ALL
  SELECT c.otro, r.camino || c.otro, r.saltos + 1
  FROM ruta r
  JOIN (SELECT usuario_a AS uno, usuario_b AS otro FROM conexion
        UNION ALL
        SELECT usuario_b, usuario_a FROM conexion) c ON c.uno = r.actual
  WHERE r.saltos < 6 AND NOT c.otro = ANY(r.camino)
)
SELECT camino, saltos FROM ruta
WHERE actual = 'usr_paula24'
ORDER BY saltos LIMIT 1;
```

Además de ser más larga, esta consulta hace más trabajo: en cada nivel vuelve a cruzar la tabla de conexiones y explora todos los caminos de hasta seis pasos antes de elegir el más corto. `shortestPath` de Neo4j, en cambio, avanza desde los dos extremos a la vez y se detiene en cuanto los caminos se encuentran.

La diferencia crece con la profundidad. En el libro *Neo4j in Action* (2014) se comparan MySQL y Neo4j buscando los amigos de los amigos en una red de un millón de personas con unos 50 contactos cada una:

Tabla 10. Tiempo para encontrar contactos a distinta profundidad en una red de un millón de personas, según el libro *Neo4j in Action* (2014, cap. 1).

Profundidad	MySQL (s)	Neo4j (s)	Personas encontradas
2	0,016	0,010	2 500
3	30,267	0,168	110 000
4	1 543,505	1,359	600 000
5	no terminó	2,132	800 000

A dos pasos los dos responden al instante. Desde el tercero, cada `JOIN` más multiplica el trabajo de la base relacional, mientras que Neo4j solo recorre los contactos que encuentra. Las cifras exactas dependen del equipo y de cómo se configure cada base, y quienes han intentado repetir el experimento no siempre han obtenido una diferencia tan grande. Por eso lo tomamos como una ilustración de la tendencia y no como una medida exacta.

Preguntas que combinan varios tipos de relación

La consulta de candidatos (Sección 6.4) recorre cinco tipos de relación: **REQUIERE**, **TIENE_HABILIDAD**, **CONECTA_CON**, **TRABAJA_EN** y **PUBLICA**. En Cypher el patrón se lee casi igual que la pregunta. En SQL serían al menos seis tablas unidas con `JOIN`, y el sentido de la consulta se pierde entre las condiciones de unión.

Un esquema que cambia

Durante el desarrollo agregamos dieciséis tipos de relación, como las alianzas entre empresas y todas las solicitudes pendientes. En Neo4j bastó con empezar a crearlas: las consultas que ya existían no se vieron afectadas porque solo recorren los tipos que nombran. En una base relacional, cada una habría sido una tabla nueva y una migración de la base.

Recomendaciones a partir de una persona

Las recomendaciones parten de un nodo y exploran sus alrededores. Su costo depende del tamaño de ese vecindario, no del tamaño de la red. Es el caso de uso más común de las bases de grafos: sugerencias de contactos, detección de fraude o rutas logísticas.

7.3. Dónde no gana el grafo

Sumas y reportes sobre todos los datos

"Las habilidades más demandadas" no recorre la red: cuenta requisitos y los agrupa. En SQL es un `GROUP BY` igual de simple que el Cypher de la Sección 6.6:

```sql
SELECT h.nombre, COUNT(*) AS ofertas
FROM requisito r
JOIN habilidad h ON h.hid = r.hid
JOIN oferta o    ON o.oid = r.oid
WHERE o.estado = 'abierta'
GROUP BY h.nombre
ORDER BY ofertas DESC;
```

Para este tipo de preguntas el grafo no aporta nada, y las bases relacionales tienen décadas de optimización y herramientas de reportes alrededor.

Reglas de integridad

Una base relacional hace cumplir reglas por sí sola: claves foráneas, campos obligatorios (`NOT NULL`) y condiciones (`CHECK`). La edición gratuita de Neo4j solo garantiza que un valor no se repita. En WheelsOn, reglas como "una oferta pertenece a una sola empresa", "cada empresa tiene un solo administrador" o "no se acepta a más personas que cupos" tuvimos que programarlas nosotros y probarlas a mano para asegurarnos de que los datos siguieran siendo consistentes.

Datos con estructura fija y pocas relaciones

Facturas, inventarios o registros contables tienen siempre los mismos campos y casi no se recorren como red. En esos casos, las tablas son más simples, SQL es un estándar que casi todo el mundo conoce y la base relacional es la mejor opción.

Algoritmos sobre la red completa

PageRank o la detección de comunidades no son recorridos cortos: procesan todos los nodos varias veces. Ninguno de los dos modelos los resuelve con una consulta normal; hace falta una biblioteca especializada o, como en WheelsOn, calcularlos fuera de la base (Sección 6.8).

Estandarización

SQL es un estándar desde 1986 y casi todas las bases relacionales lo hablan. Los lenguajes de grafos apenas se están unificando: el estándar ISO GQL, basado en buena parte en Cypher, se publicó en 2024.

7.4. Resumen

Tabla 11. Qué modelo conviene según el tipo de pregunta.

Escenario	Conviene	Ejemplo en WheelsOn
Caminos de longitud desconocida	Grafo	Ruta más corta hacia una empresa.
Preguntas que cruzan muchos tipos de relación	Grafo	Candidatos por competencias y contactos.
Recomendaciones desde una persona o un proyecto	Grafo	Contactos, empleos y colaboradores sugeridos.
Esquema que cambia con frecuencia	Grafo	Alianzas y solicitudes agregadas sobre la marcha.
Conteos y reportes sobre todo el conjunto	Cualquiera	Habilidades más demandadas.
Reglas de integridad estrictas	Relacional	Cupos de una oferta y administrador único.
Datos tabulares con pocas relaciones	Relacional	No hay en WheelsOn: sería el caso de pagos o facturación.
Algoritmos sobre la red completa	Biblioteca especializada	Influencia y comunidades.

Para elegir entre los dos hay que mirar qué tipo de preguntas se le van a hacer a los datos. Si recorren relaciones a una profundidad que no se conoce de antemano, conviene el grafo. Si suman, filtran o validan registros de estructura fija, la base relacional es más simple y más segura. Muchos sistemas reales usan las dos: una relacional para las transacciones y un grafo para las recomendaciones.

8. Conclusiones

- Implementamos todas las funcionalidades del enunciado guardando la red completa en Neo4j, y respondimos sus seis preguntas con consultas Cypher que también usa la aplicación.
- Guardar en las relaciones los datos de cada vínculo, como el cargo, la etapa de una postulación o la prioridad de un requisito, fue la decisión de modelado que más simplificó las consultas.
- Separar lo pendiente de lo confirmado en tipos de relación distintos evitó que las rutas, las recomendaciones y la analítica tuvieran que filtrar estados en cada consulta.
- El grafo nos dio consultas cortas y legibles para caminos y recomendaciones, pero nos obligó a programar y probar a mano reglas de integridad que una base relacional habría hecho cumplir por sí sola.
- De la investigación concluimos que un grafo conviene cuando las preguntas recorren relaciones de profundidad variable, y una base relacional cuando se trata de sumar, filtrar y validar datos con estructura fija.

9. Referencias

- Neo4j, Inc. *Cypher Manual*, versión 5. `https://neo4j.com/docs/cypher-manual/5/`
- Neo4j, Inc. *Cypher Manual*: restricciones y su disponibilidad por edición. `https://neo4j.com/docs/cypher-manual/5/constraints/`
- ISO/IEC 39075:2024. *Information technology — Database languages — GQL*.
- Page, L., Brin, S., Motwani, R. y Winograd, T. (1999). *The PageRank Citation Ranking: Bringing Order to the Web*. Stanford InfoLab.

A. Script de inicio de la base de datos

Este es el script completo con el que creamos las restricciones y los índices, guardado en `scripts/constraints.cypher`. Los dos scripts de datos suman cerca de mil líneas de `CREATE` con el mismo formato que mostramos en la Sección 5, y están en la misma carpeta del repositorio.

```
// ============================================================================
// WheelsOn - Constraints e Indices para Neo4j (Cypher)
// ============================================================================

// Restricciones de unicidad (Unique Constraints)
CREATE CONSTRAINT usuario_uid IF NOT EXISTS
```

```
  FOR (u:Usuario) REQUIRE u.uid IS UNIQUE;

CREATE CONSTRAINT usuario_email IF NOT EXISTS
  FOR (u:Usuario) REQUIRE u.email IS UNIQUE;

CREATE CONSTRAINT empresa_eid IF NOT EXISTS
  FOR (e:Empresa) REQUIRE e.eid IS UNIQUE;

CREATE CONSTRAINT empresa_nombre IF NOT EXISTS
  FOR (e:Empresa) REQUIRE e.nombre IS UNIQUE;

CREATE CONSTRAINT habilidad_hid IF NOT EXISTS
  FOR (h:Habilidad) REQUIRE h.hid IS UNIQUE;

CREATE CONSTRAINT habilidad_nombre IF NOT EXISTS
  FOR (h:Habilidad) REQUIRE h.nombre IS UNIQUE;

CREATE CONSTRAINT proyecto_pid IF NOT EXISTS
  FOR (p:Proyecto) REQUIRE p.pid IS UNIQUE;

CREATE CONSTRAINT oferta_oid IF NOT EXISTS
  FOR (o:Oferta) REQUIRE o.oid IS UNIQUE;

CREATE CONSTRAINT publicacion_pubid IF NOT EXISTS
  FOR (p:Publicacion) REQUIRE p.pubid IS UNIQUE;

CREATE CONSTRAINT avance_aid IF NOT EXISTS
  FOR (a:Avance) REQUIRE a.aid IS UNIQUE;

CREATE CONSTRAINT notificacion_nid IF NOT EXISTS
  FOR (n:Notificacion) REQUIRE n.nid IS UNIQUE;

// Indices de rendimiento para busquedas frecuentes
CREATE INDEX usuario_nombres IF NOT EXISTS
  FOR (u:Usuario) ON (u.nombre, u.apellido);

CREATE INDEX empresa_industria IF NOT EXISTS
  FOR (e:Empresa) ON (e.industria);

CREATE INDEX oferta_estado IF NOT EXISTS
  FOR (o:Oferta) ON (o.estado);

CREATE INDEX habilidad_categoria IF NOT EXISTS
  FOR (h:Habilidad) ON (h.categoria);

CREATE INDEX proyecto_estado IF NOT EXISTS
  FOR (p:Proyecto) ON (p.estado);
```

B. Script de consultas

Este es el archivo completo con las consultas que usa la aplicación (`scripts/queries.cypher`).

```
// ============================================================================
// WHEELSON - RED PROFESIONAL BASADA EN BASE DE DATOS DE GRAFOS
// SCRIPT DE CONSULTAS AVANZADAS EN LENGUAJE CYPHER (FASE 6)
// ============================================================================

// ----------------------------------------------------------------------------
// CONSULTA 1: Ruta mas corta entre dos usuarios (Shortest Path / Grados de Separacion)
// Encuentra la cadena minima de intermediarios entre dos profesionales en la red.
// ----------------------------------------------------------------------------
MATCH (u1:Usuario {uid: $uid1}), (u2:Usuario {uid: $uid2})
MATCH p = shortestPath((u1)-[:CONECTA_CON*..6]-(u2))
RETURN [n IN nodes(p) | {uid: n.uid, nombre: n.nombre, apellido: n.apellido, titulo: n.titulo_profesional
     }] AS ruta_nodos,
       length(p) AS grados_separacion;

// ----------------------------------------------------------------------------
// CONSULTA 2: Contactos en comun entre dos usuarios (Mutual Connections)
// Identifica profesionales compartidos conectados directamente con ambos usuarios.
// ----------------------------------------------------------------------------
MATCH (u1:Usuario {uid: $uid1})-[:CONECTA_CON]-(comun:Usuario)-[:CONECTA_CON]-(u2:Usuario {uid: $uid2})
WHERE u1 <> u2
RETURN DISTINCT comun.uid AS uid,
       comun.nombre + ' ' + comun.apellido AS nombre_completo,
       comun.titulo_profesional AS titulo,
       comun.ubicacion AS ubicacion
ORDER BY nombre_completo ASC;

// ----------------------------------------------------------------------------
// CONSULTA 3: Usuarios con habilidades tecnicas similares (Similitud por Habilidades)
// Recomienda colegas profesionales con base en la interseccion de habilidades que dominan.
// ----------------------------------------------------------------------------
MATCH (u1:Usuario {uid: $uid1})-[:TIENE_HABILIDAD]->(h:Habilidad)<-[:TIENE_HABILIDAD]-(u2:Usuario)
WHERE u1 <> u2
WITH u2, count(h) AS total_compartidas, collect(h.nombre) AS habilidades_comunes
ORDER BY total_compartidas DESC, u2.nombre ASC
RETURN u2.uid AS uid,
       u2.nombre + ' ' + u2.apellido AS nombre_completo,
       u2.titulo_profesional AS titulo,
       total_compartidas,
       habilidades_comunes
LIMIT $limit;

// ----------------------------------------------------------------------------
// CONSULTA 4: Ruta usuario -> empresa (Networking para Referidos)
// Traza el camino mas corto desde un usuario hacia una empresa a traves de contactos y colegas.
// ----------------------------------------------------------------------------
MATCH (u:Usuario {uid: $uid}), (e:Empresa {eid: $eid})
MATCH p = shortestPath((u)-[:CONECTA_CON|TRABAJA_EN*..6]-(e))
WHERE all(r IN relationships(p) WHERE type(r) <> 'TRABAJA_EN' OR coalesce(r.actual, true))
RETURN [n IN nodes(p) | {
    tipo: labels(n)[0],
    id: coalesce(n.uid, n.eid),
```

```
    nombre: CASE WHEN n:Usuario THEN n.nombre + ' ' + n.apellido ELSE n.nombre END
}] AS pasos_ruta,
length(p) AS longitud_camino;

// ------------------------------------------------------------------------------
// CONSULTA 5: Habilidades mas demandadas por el mercado laboral
// Agrupa y contabiliza las habilidades requeridas por vacantes abiertas, discriminando obligatorias.
// ------------------------------------------------------------------------------
MATCH (h:Habilidad)<-[r:REQUIERE]-(o:Oferta {estado: 'abierta'})
RETURN h.hid AS hid,
       h.nombre AS habilidad,
       h.categoria AS categoria,
       count(o) AS total_ofertas,
       count(CASE WHEN r.prioridad = 'obligatoria' THEN 1 END) AS obligatorias,
       count(CASE WHEN r.prioridad = 'deseable' THEN 1 END) AS deseables
ORDER BY total_ofertas DESC, obligatorias DESC
LIMIT $limit;

// ------------------------------------------------------------------------------
// CONSULTA 6: Candidatos recomendados para una oferta segun competencias y conexiones
// Debe cumplir al menos un requisito; los contactos en la empresa suben su puesto.
// Excluye a quienes ya trabajan en la empresa que publica la oferta.
// ------------------------------------------------------------------------------
MATCH (o:Oferta {oid: $oid})-[r:REQUIERE]->(h:Habilidad)<-[:TIENE_HABILIDAD]-(u:Usuario)
WHERE NOT EXISTS {
    MATCH (u)-[t:TRABAJA_EN]->(:Empresa)-[:PUBLICA]->(o)
    WHERE coalesce(t.actual, true)
}
WITH o, u,
     count(DISTINCT h) AS total_coincidencias,
     collect(DISTINCT h.nombre) AS habilidades_match,
     count(DISTINCT CASE WHEN r.prioridad = 'obligatoria' THEN h END) AS obligatorias_cubiertas
OPTIONAL MATCH (u)-[:CONECTA_CON]-(c:Usuario)-[t:TRABAJA_EN]->(:Empresa)-[:PUBLICA]->(o)
WHERE coalesce(t.actual, true)
WITH o, u, total_coincidencias, habilidades_match, obligatorias_cubiertas,
     collect(DISTINCT c.nombre + ' ' + c.apellido) AS contactos
RETURN u.uid AS uid,
       u.nombre + ' ' + u.apellido AS nombre_completo,
       total_coincidencias, obligatorias_cubiertas, habilidades_match, contactos,
       total_coincidencias * 3 + obligatorias_cubiertas * 2 + size(contactos) * 2 AS score,
       EXISTS { (u)-[:POSTULA_A]->(o) } AS ya_postulado
ORDER BY score DESC, total_coincidencias DESC, nombre_completo ASC
LIMIT $limit;

// ------------------------------------------------------------------------------
// CONSULTA 7: Recomendar contactos
// Personas sin conexion directa, puntuadas por contactos, habilidades y empresas en comun.
// ------------------------------------------------------------------------------
MATCH (yo:Usuario {uid: $uid}), (otro:Usuario)
WHERE otro <> yo AND NOT (yo)-[:CONECTA_CON]-(otro)
OPTIONAL MATCH (yo)-[:CONECTA_CON]-(amigo:Usuario)-[:CONECTA_CON]-(otro)
WITH yo, otro, count(DISTINCT amigo) AS amigos_comun,
     collect(DISTINCT amigo.nombre + ' ' + amigo.apellido) AS nombres_amigos
OPTIONAL MATCH (yo)-[:TIENE_HABILIDAD]->(h:Habilidad)<-[:TIENE_HABILIDAD]-(otro)
WITH yo, otro, amigos_comun, nombres_amigos, count(DISTINCT h) AS habs_comun,
     collect(DISTINCT h.nombre) AS nombres_habs
OPTIONAL MATCH (yo)-[t1:TRABAJA_EN]->(e:Empresa)<-[t2:TRABAJA_EN]-(otro)
WHERE coalesce(t1.actual, true) AND coalesce(t2.actual, true)
WITH otro, amigos_comun, nombres_amigos, habs_comun, nombres_habs, count(DISTINCT e) AS emps_comun
```

```
WITH otro, amigos_comun, nombres_amigos, habs_comun, nombres_habs, emps_comun,
     amigos_comun * 4 + habs_comun * 2 + emps_comun * 3 AS score
WHERE score > 0
RETURN otro, score, amigos_comun, nombres_amigos[..3], habs_comun, nombres_habs, emps_comun
ORDER BY score DESC, amigos_comun DESC, otro.nombre ASC
LIMIT $limit;

// ------------------------------------------------------------------------------
// CONSULTA 8: Recomendar empleos
// Ofertas abiertas sin postulacion, con el porcentaje de habilidades que ya se cumplen.
// ------------------------------------------------------------------------------
MATCH (u:Usuario {uid: $uid}), (o:Oferta {estado: 'abierta'})
WHERE NOT (u)-[:POSTULA_A]->(o)
MATCH (o)-[r:REQUIERE]->(h:Habilidad)
WITH u, o, r, h, EXISTS { (u)-[:TIENE_HABILIDAD]->(h) } AS la_tiene
WITH o,
     count(h) AS total_requisitos,
     [n IN collect(CASE WHEN la_tiene THEN h.nombre END) WHERE n IS NOT NULL] AS coinciden,
     [n IN collect(CASE WHEN NOT la_tiene THEN h.nombre END) WHERE n IS NOT NULL] AS faltan,
     count(CASE WHEN r.prioridad = 'obligatoria' THEN 1 END) AS total_obligatorias,
     count(CASE WHEN r.prioridad = 'obligatoria' AND la_tiene THEN 1 END) AS obligatorias_cubiertas
WHERE size(coinciden) > 0
OPTIONAL MATCH (e:Empresa)-[:PUBLICA]->(o)
RETURN o, e, total_requisitos, coinciden, faltan, total_obligatorias, obligatorias_cubiertas,
       toFloat(size(coinciden)) / total_requisitos AS proporcion
ORDER BY proporcion DESC, obligatorias_cubiertas DESC, o.fecha_publicacion DESC
LIMIT $limit;

// ------------------------------------------------------------------------------
// CONSULTA 9: Recomendar colaboradores para un proyecto
// Personas externas con habilidades del stack del proyecto o contactos en su equipo.
// ------------------------------------------------------------------------------
MATCH (p:Proyecto {pid: $pid}), (u:Usuario)
WHERE NOT (u)-[:PARTICIPA_EN]->(p)
OPTIONAL MATCH (u)-[:TIENE_HABILIDAD]->(h:Habilidad)
WHERE any(t IN coalesce(p.tecnologias, []) WHERE toLower(h.nombre) CONTAINS toLower(t))
WITH p, u, collect(DISTINCT h.nombre) AS habilidades
OPTIONAL MATCH (u)-[:CONECTA_CON]-(m:Usuario)-[:PARTICIPA_EN]->(p)
WITH u, habilidades, collect(DISTINCT m.nombre + ' ' + m.apellido) AS companeros
WITH u, habilidades, companeros, size(habilidades) * 5 + size(companeros) * 3 AS score
WHERE score > 0
RETURN u, score, habilidades, companeros
ORDER BY score DESC, size(habilidades) DESC, size(companeros) DESC, u.nombre ASC
LIMIT $limit;

// ------------------------------------------------------------------------------
// CONSULTA 10: Recomendar talento para una empresa
// Personas externas con habilidades pedidas en sus ofertas abiertas o contactos internos.
// ------------------------------------------------------------------------------
MATCH (e:Empresa {eid: $eid})
OPTIONAL MATCH (e)-[:PUBLICA]->(:Oferta {estado: 'abierta'})-[:REQUIERE]->(req:Habilidad)
WITH e, collect(DISTINCT req) AS requeridas
MATCH (u:Usuario)
WHERE NOT EXISTS { (u)-[t:TRABAJA_EN]->(e) WHERE coalesce(t.actual, true) }
OPTIONAL MATCH (u)-[:TIENE_HABILIDAD]->(h:Habilidad)
WHERE h IN requeridas
WITH e, u, collect(DISTINCT h.nombre) AS habilidades
OPTIONAL MATCH (u)-[:CONECTA_CON]-(c:Usuario)-[t:TRABAJA_EN]->(e)
WHERE coalesce(t.actual, true)
```

```
WITH u, habilidades, count(DISTINCT c) AS conexiones
WITH u, habilidades, conexiones, size(habilidades) * 4 + conexiones * 3 AS score
WHERE score > 0
RETURN u, score, habilidades, conexiones
ORDER BY score DESC, size(habilidades) DESC, u.nombre ASC
LIMIT $limit;

// ------------------------------------------------------------------------
// CONSULTA 11: Red de conexiones para la analitica
// Devuelve las aristas; centralidad, PageRank y comunidades se calculan con ellas en Python.
// ------------------------------------------------------------------------
MATCH (a:Usuario)-[:CONECTA_CON]->(b:Usuario)
RETURN a.uid, b.uid;

// ------------------------------------------------------------------------
// CONSULTA 12: Relaciones del grafo interactivo
// ------------------------------------------------------------------------
MATCH (a)-[r:CONECTA_CON|TRABAJA_EN|TIENE_HABILIDAD|PARTICIPA_EN|PUBLICA|REQUIERE|POSTULA_A|ADMINISTRA|
    LIDERA]->(b)
RETURN coalesce(a.uid, a.eid, a.hid, a.pid, a.oid) AS origen,
       coalesce(b.uid, b.eid, b.hid, b.pid, b.oid) AS destino,
       type(r) AS tipo,
       properties(r) AS propiedades;

// ------------------------------------------------------------------------
// CONSULTA 13: Camino de un usuario a una oferta
// Pasa por contactos y empleos hasta la empresa que publica la oferta.
// ------------------------------------------------------------------------
MATCH (u:Usuario {uid: $uid}), (o:Oferta {oid: $oid})
MATCH p = shortestPath((u)-[:CONECTA_CON|TRABAJA_EN|PUBLICA*..7]-(o))
WHERE all(r IN relationships(p) WHERE type(r) <> 'TRABAJA_EN' OR coalesce(r.actual, true))
RETURN [n IN nodes(p) | {
    tipo: labels(n)[0],
    id: coalesce(n.uid, n.eid, n.oid),
    nombre: CASE
        WHEN n:Usuario THEN n.nombre + ' ' + n.apellido
        WHEN n:Oferta THEN n.titulo
        ELSE n.nombre
    END
}] AS pasos_ruta,
[r IN relationships(p) | type(r)] AS relaciones,
length(p) AS longitud_camino;

// ------------------------------------------------------------------------
// CONSULTA 14: Proyectos que comparten tecnologias o miembros relacionados
// Miembros relacionados: personas de ambos equipos conectadas entre si.
// ------------------------------------------------------------------------
MATCH (p:Proyecto {pid: $pid}), (otro:Proyecto)
WHERE otro <> p
WITH p, otro,
     [t IN coalesce(p.tecnologias, [])
      WHERE toLower(t) IN [x IN coalesce(otro.tecnologias, []) | toLower(x)]] AS tecnologias
OPTIONAL MATCH (p)<-[:PARTICIPA_EN]-(m:Usuario)-[:PARTICIPA_EN]->(otro)
WITH p, otro, tecnologias, collect(DISTINCT m) AS comunes
OPTIONAL MATCH (p)<-[:PARTICIPA_EN]-(a:Usuario)-[:CONECTA_CON]-(b:Usuario)-[:PARTICIPA_EN]->(otro)
WHERE NOT a IN comunes AND NOT b IN comunes
WITH otro, tecnologias, comunes,
     collect(DISTINCT a.nombre + ' ' + a.apellido + ' - ' + b.nombre + ' ' + b.apellido) AS conectados
WITH otro, tecnologias, conectados,
```

```
     [m IN comunes | m.nombre + ' ' + m.apellido] AS miembros_comunes,
     size(tecnologias) * 2 + size(comunes) * 3 + size(conectados) AS score
WHERE score > 0
RETURN otro.pid AS pid, otro.nombre AS nombre, tecnologias, miembros_comunes, conectados, score
ORDER BY score DESC, nombre ASC
LIMIT $limit;

// ------------------------------------------------------------------------------
// CONSULTA 15: Lo que administra un usuario
// Empresas, proyectos y ofertas sobre los que tiene permiso de gestion.
// ------------------------------------------------------------------------------
MATCH (u:Usuario {uid: $uid})
OPTIONAL MATCH (u)-[:ADMINISTRA]->(e:Empresa)
OPTIONAL MATCH (e)-[:PUBLICA]->(o:Oferta)
OPTIONAL MATCH (u)-[:LIDERA]->(p:Proyecto)
RETURN u.rol AS rol,
       collect(DISTINCT e.nombre) AS empresas,
       collect(DISTINCT o.titulo) AS ofertas,
       collect(DISTINCT p.nombre) AS proyectos;

// ------------------------------------------------------------------------------
// CONSULTA 16: Dueno de cada empresa y proyecto
// ------------------------------------------------------------------------------
MATCH (u:Usuario)-[r:ADMINISTRA|LIDERA]->(recurso)
RETURN u.nombre + ' ' + u.apellido AS dueno,
       type(r) AS relacion,
       labels(recurso)[0] AS tipo,
       recurso.nombre AS nombre
ORDER BY tipo, nombre;

// ------------------------------------------------------------------------------
// Cupos y postulaciones de un empleo
// Aceptar una postulación crea (:Usuario)-[:TRABAJA_EN]->(:Empresa) con el título del
// empleo como cargo. Al llenarse los cupos, el empleo se cierra y quienes seguían en
// proceso quedan como 'no_seleccionada'.
// ------------------------------------------------------------------------------
MATCH (e:Empresa)-[:PUBLICA]->(o:Oferta {oid: $oid})
RETURN o.titulo AS empleo, e.nombre AS empresa, o.estado AS estado,
       coalesce(o.cupos, 1) AS cupos,
       COUNT { (:Usuario)-[:POSTULA_A]->(o) } AS postulaciones,
       COUNT { (:Usuario)-[r:POSTULA_A]->(o) WHERE r.estado = 'aceptada' } AS contratados;
```